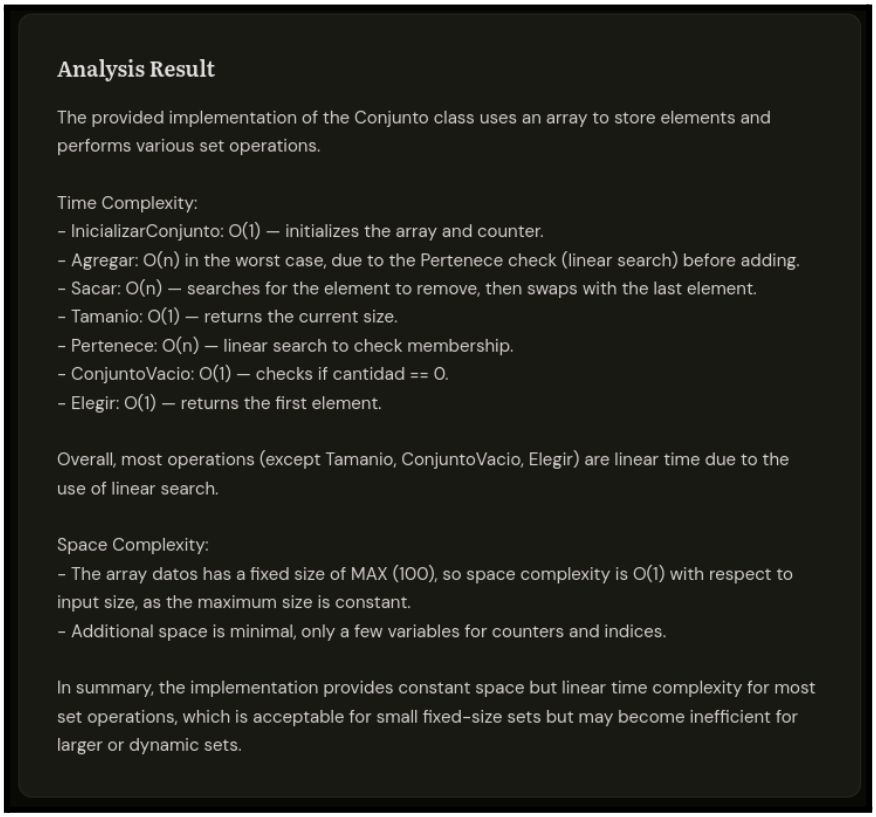
Métodos	Estático	Dinámico
Apilar()	Temp: O(n) Espac. O(n)	
Desapilar()	Temp: O(1) Espac. O(n)	Pila: Temp: O(1) Espac. O(n)
Tope()	Temp: O(1) Espac. O(n)	
PIla Vacía	Temp: O(1) Espac. O(n)	
TDA Conjunto	Temp: O(N) Espac. O(n)	Temp: O(n) Espac. O(n)
TDA Cola	Temp: O(1) Espac. O(n)	Temp: O(1) Espac. O(n)
TDA Cola Prioridad	Temp: O(n) Espac. O(n)	Temp: O(n) Espac. O(n)
TDA Diccionario simple	Temp: O(n) Espac. O(n)	Temp: O(n) Espac. O(n)
TDA Diccionario múltiple	Temp: O(n) Espac. O(n)	Temp: O(n) Espac. O(n)

Pruebas realizadas en Big-O Calculator Estáticos:

- **TDA Conjunto Estático**



Analysis Result

The provided implementation of the Conjunto class uses an array to store elements and performs various set operations.

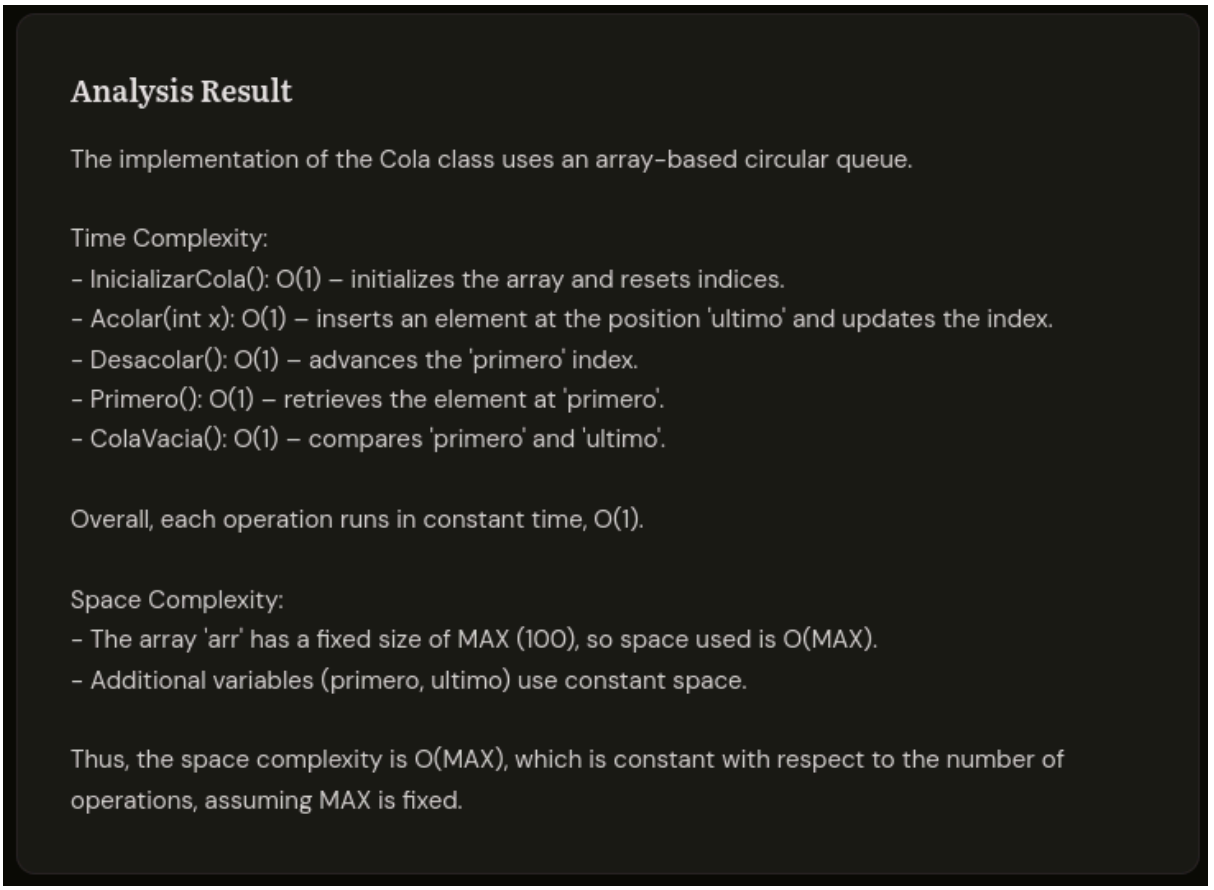
Time Complexity:
- InicializarConjunto: O(1) — initializes the array and counter.
- Agregar: O(n) in the worst case, due to the Pertenece check (linear search) before adding.
- Sacar: O(n) — searches for the element to remove, then swaps with the last element.
- Tamanio: O(1) — returns the current size.
- Pertenece: O(n) — linear search to check membership.
- ConjuntoVacio: O(1) — checks if cantidad == 0.
- Elegir: O(1) — returns the first element.

Overall, most operations (except Tamanio, ConjuntoVacio, Elegir) are linear time due to the use of linear search.

Space Complexity:
- The array datos has a fixed size of MAX (100), so space complexity is O(1) with respect to input size, as the maximum size is constant.
- Additional space is minimal, only a few variables for counters and indices.

In summary, the implementation provides constant space but linear time complexity for most set operations, which is acceptable for small fixed-size sets but may become inefficient for larger or dynamic sets.

- **TDA Cola Estático**



Analysis Result

The implementation of the Cola class uses an array-based circular queue.

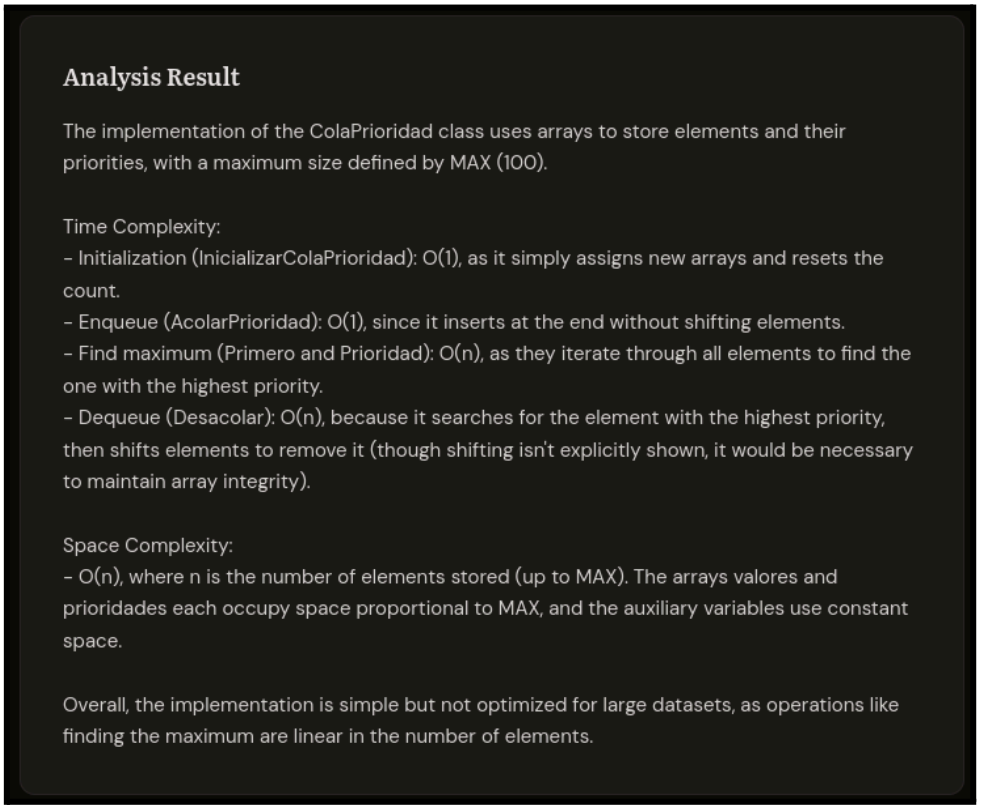
Time Complexity:
- InicializarCola(): O(1) – initializes the array and resets indices.
- Acolar(int x): O(1) – inserts an element at the position 'ultimo' and updates the index.
- Desacolar(): O(1) – advances the 'primero' index.
- Primero(): O(1) – retrieves the element at 'primero'.
- ColaVacia(): O(1) – compares 'primero' and 'ultimo'.

Overall, each operation runs in constant time, O(1).

Space Complexity:
- The array 'arr' has a fixed size of MAX (100), so space used is O(MAX).
- Additional variables (primero, ultimo) use constant space.

Thus, the space complexity is O(MAX), which is constant with respect to the number of operations, assuming MAX is fixed.

- **TDA cola con prioridad:**



Analysis Result

The implementation of the ColaPrioridad class uses arrays to store elements and their priorities, with a maximum size defined by MAX (100).

Time Complexity:
- Initialization (InicializarColaPrioridad): O(1), as it simply assigns new arrays and resets the count.
- Enqueue (AcolarPrioridad): O(1), since it inserts at the end without shifting elements.
- Find maximum (Primero and Prioridad): O(n), as they iterate through all elements to find the one with the highest priority.
- Dequeue (Desacolar): O(n), because it searches for the element with the highest priority, then shifts elements to remove it (though shifting isn't explicitly shown, it would be necessary to maintain array integrity).

Space Complexity:
- O(n), where n is the number of elements stored (up to MAX). The arrays valores and prioridades each occupy space proportional to MAX, and the auxiliary variables use constant space.

Overall, the implementation is simple but not optimized for large datasets, as operations like finding the maximum are linear in the number of elements.

- **TDA diccionario simple**

Analysis Result

The implementation of DiccionarioSimple uses arrays to store keys and values, with a fixed maximum size (MAX = 100).

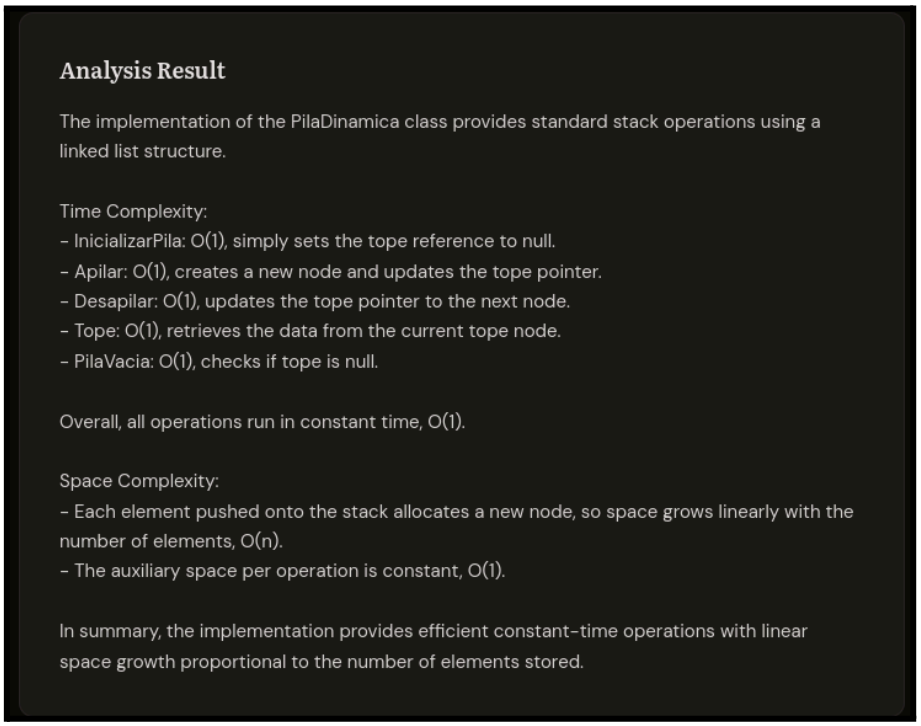
Time Complexity:
- InicializarDiccionario: O(1), as it just initializes arrays and counter.
- buscarIndice: O(n), where n is the number of stored elements (up to MAX), since it performs a linear search.
- Agregar: O(n) in the worst case, because it calls buscarIndice to check if the key exists, which is linear, and then potentially adds a new element.
- Eliminar: O(n), due to buscarIndice to find the key, and then shifting or replacing the element at the found index.
- Recuperar: O(n), as it calls buscarIndice to find the key.

Space Complexity:
- Uses arrays of size MAX for storing keys and values, so space complexity is O(MAX). The actual used space depends on the number of stored elements (up to MAX), but the allocated space remains constant.

Overall, the implementation is simple but not optimized for large datasets, as most operations are linear in the number of stored elements.

Pruebas de TDA dinámicos realizadas en Big-O Calculator:

- **TDA pila dinámica**



Analysis Result

The implementation of the PilaDinamica class provides standard stack operations using a linked list structure.

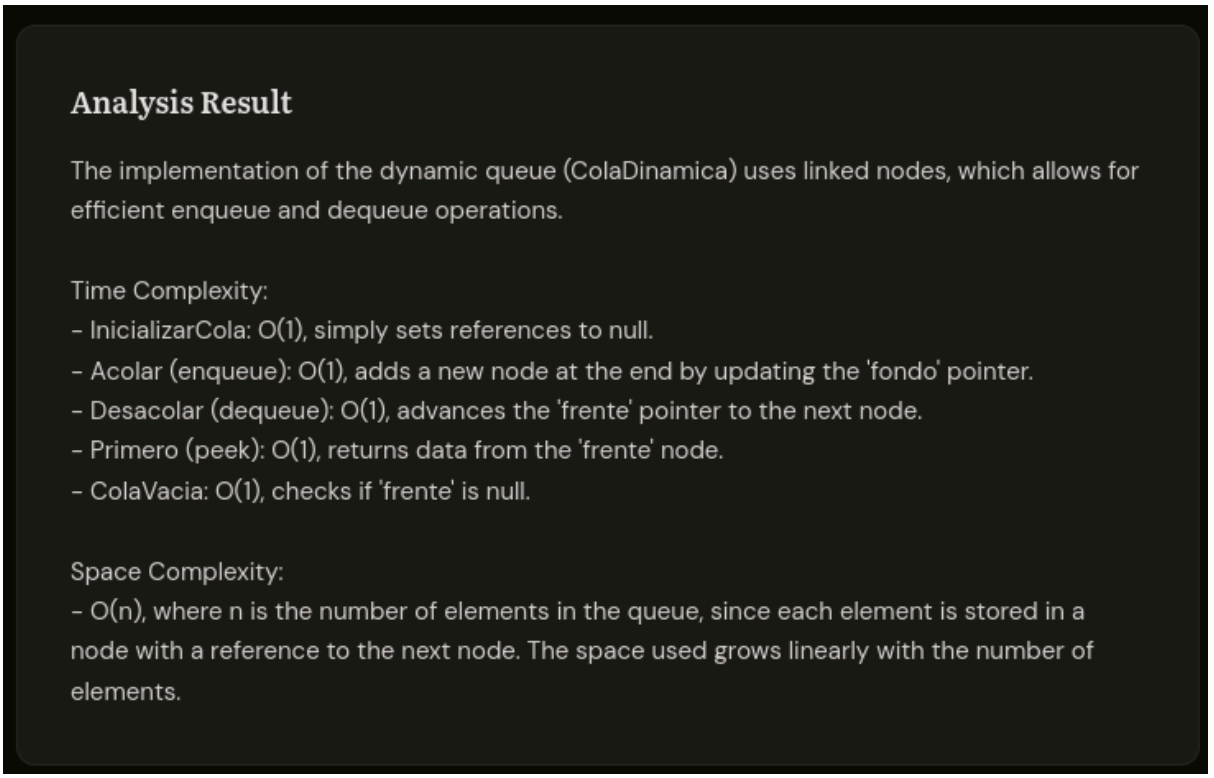
Time Complexity:
- InicializarPila: O(1), simply sets the tope reference to null.
- Apilar: O(1), creates a new node and updates the tope pointer.
- Desapilar: O(1), updates the tope pointer to the next node.
- Tope: O(1), retrieves the data from the current tope node.
- PilaVacia: O(1), checks if tope is null.

Overall, all operations run in constant time, O(1).

Space Complexity:
- Each element pushed onto the stack allocates a new node, so space grows linearly with the number of elements, O(n).
- The auxiliary space per operation is constant, O(1).

In summary, the implementation provides efficient constant-time operations with linear space growth proportional to the number of elements stored.

- **TDA Cola Dinámica**



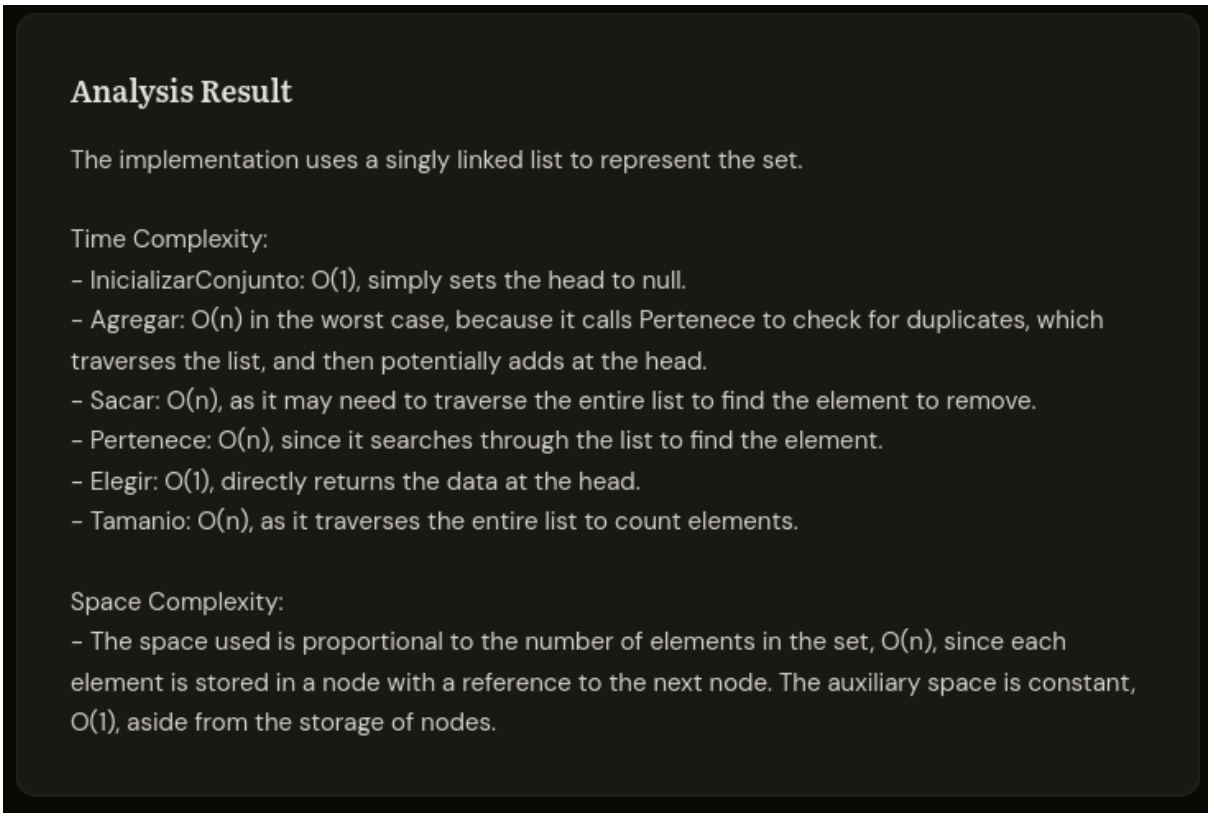
Analysis Result

The implementation of the dynamic queue (ColaDinamica) uses linked nodes, which allows for efficient enqueue and dequeue operations.

Time Complexity:
- InicializarCola: O(1), simply sets references to null.
- Acolar (enqueue): O(1), adds a new node at the end by updating the 'fondo' pointer.
- Desacolar (dequeue): O(1), advances the 'frente' pointer to the next node.
- Primero (peek): O(1), returns data from the 'frente' node.
- ColaVacia: O(1), checks if 'frente' is null.

Space Complexity:
- O(n), where n is the number of elements in the queue, since each element is stored in a node with a reference to the next node. The space used grows linearly with the number of elements.

- **TDA Conjunto dinámico**



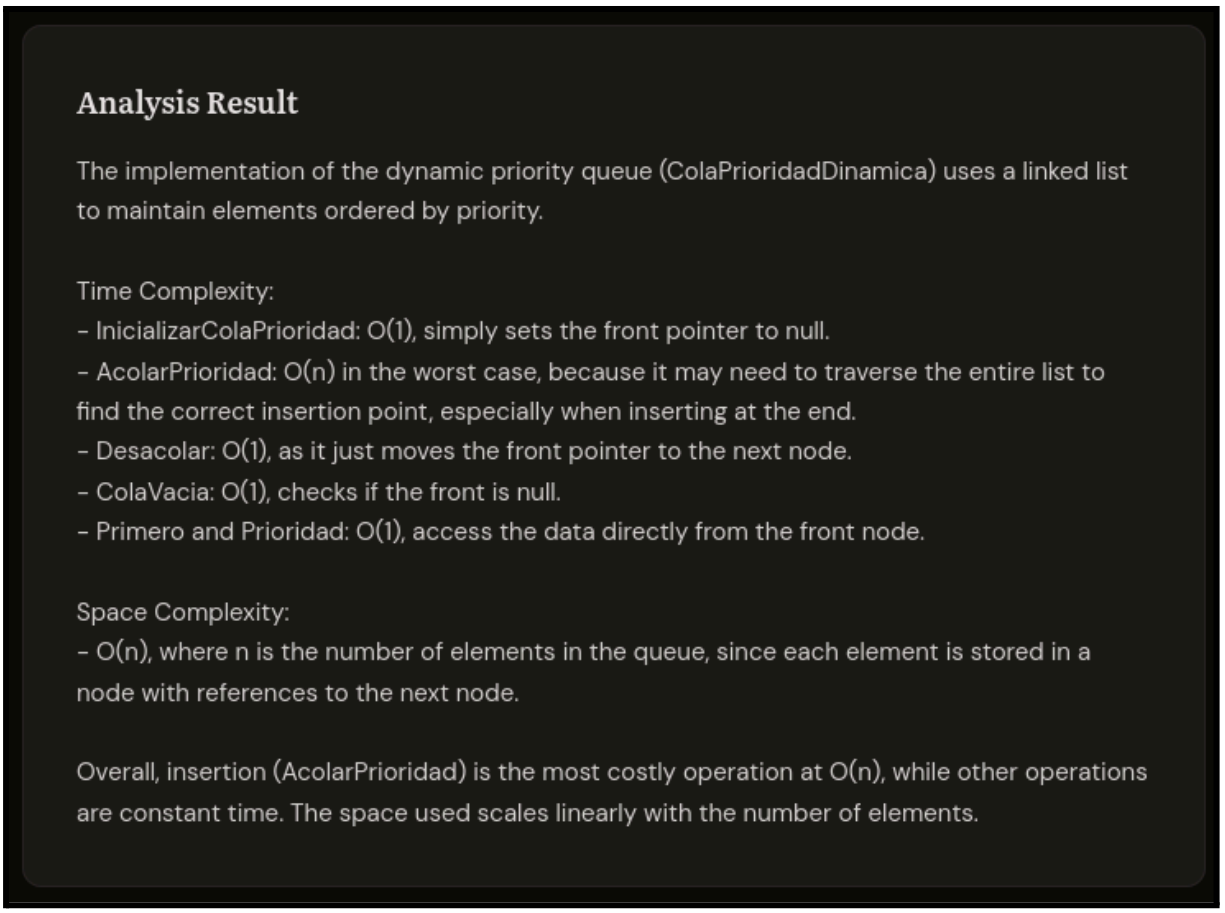
Analysis Result

The implementation uses a singly linked list to represent the set.

Time Complexity:
- InicializarConjunto: O(1), simply sets the head to null.
- Agregar: O(n) in the worst case, because it calls Pertenece to check for duplicates, which traverses the list, and then potentially adds at the head.
- Sacar: O(n), as it may need to traverse the entire list to find the element to remove.
- Pertenece: O(n), since it searches through the list to find the element.
- Elegir: O(1), directly returns the data at the head.
- Tamanio: O(n), as it traverses the entire list to count elements.

Space Complexity:
- The space used is proportional to the number of elements in the set, O(n), since each element is stored in a node with a reference to the next node. The auxiliary space is constant, O(1), aside from the storage of nodes.

- **TDA Cola con prioridad dinámica:**



Analysis Result

The implementation of the dynamic priority queue (ColaPrioridadDinamica) uses a linked list to maintain elements ordered by priority.

Time Complexity:
- InicializarColaPrioridad: O(1), simply sets the front pointer to null.
- AcolarPrioridad: O(n) in the worst case, because it may need to traverse the entire list to find the correct insertion point, especially when inserting at the end.
- Desacolar: O(1), as it just moves the front pointer to the next node.
- ColaVacia: O(1), checks if the front is null.
- Primero and Prioridad: O(1), access the data directly from the front node.

Space Complexity:
- O(n), where n is the number of elements in the queue, since each element is stored in a node with references to the next node.

Overall, insertion (AcolarPrioridad) is the most costly operation at O(n), while other operations are constant time. The space used scales linearly with the number of elements.

TDA diccionario simple dinámico:

Analysis Result

The implementation uses a singly linked list to store key-value pairs, where each node (NodeClave) contains a key, a reference to a linked list of values (NodeValor), and a pointer to the next NodeClave.

Time Complexity:
- InicializarDiccionario: O(1) – simply sets the head to null.
- Agregar: O(n) in the worst case, where n is the number of entries, because it may need to traverse the entire list to find if the key exists. If the key exists, updating the value is O(1). If not, inserting at the head is O(1).
- Eliminar: O(n) in the worst case, as it may need to traverse the entire list to find the key to delete.
- Recuperar: O(n) in the worst case, as it may need to traverse the list to find the key.
- Claves: O(n), since it traverses the entire list to collect all keys.

Space Complexity:
- O(n) for storing n key-value pairs, with each node containing references to the key, value, and next node. The total space grows linearly with the number of entries.

Overall, this implementation is efficient for small datasets but may become slow as the number of entries increases due to linear search times.

TDA diccionario múltiple dinámico

Analysis Result

The implementation uses linked lists to store keys and associated values.

Time Complexity:
- Initializing the dictionary (InicializarDiccionario): O(1).
- Adding a key-value pair (Agregar):
  - Searching for the key (buscarClave): O(n), where n is the number of keys.
  - If key not found, inserting at the head: O(1).
  - If key exists and value is new, inserting at the head of the value list: O(1).
- Removing a value (EliminarValor):
  - Searching for the key: O(n).
  - Traversing the value list to find the value: O(m), where m is the number of values for that key.
- Retrieving values (Recuperar):
  - Searching for the key: O(n).
  - Returning the list of values: O(m).

Overall, most operations are linear in the number of keys or values, making the average case O(n + m).

Space Complexity:
- The space used depends on the number of keys and values stored.
- Each key and value is stored in a node, so space complexity is O(n + total number of values), which is proportional to the total entries stored in the dictionary.